

CVE-2020-25179- CWE-200- CWE-497-CWE-214

 Copy 

--- Bhavesh M. Dhake

=====

**CWE-200: Exposure of Sensitive Information to an
Unauthorized Actor**

**CWE-214: Invocation of Process Using Visible Sensitive
Information**

=====

=

**CWE-200: Exposure of Sensitive Information to an
Unauthorized Actor**

Description : The product exposes sensitive information to an actor that is not explicitly authorized to have access to that information.

Demonstration 1 :

1.Setup xampp and navigate to C:/xampp/perl/bin/

save this code as vuln.pl :

```
#!/C:/xampp/perl/bin/perl.exe
use strict;
use warnings;

# Simulated valid credentials
my $valid_username = "admin";
my $valid_password = "secret123";

# Get input from command line
my $username = $ARGV[0] || "";
my $password = $ARGV[1] || "";

# VULNERABLE: Different error messages (CWE-200)
if ($username eq $valid_username) {
    if ($password eq $valid_password) {
        print "[SUCCESS] Login Successful\n";
    } else {
        print "[FAILED] Login Failed - incorrect password\n";
    }
} else {
    print "[FAILED] Login Failed - unknown username\n";
}
```

run command : perl.exe vuln.pl john 123

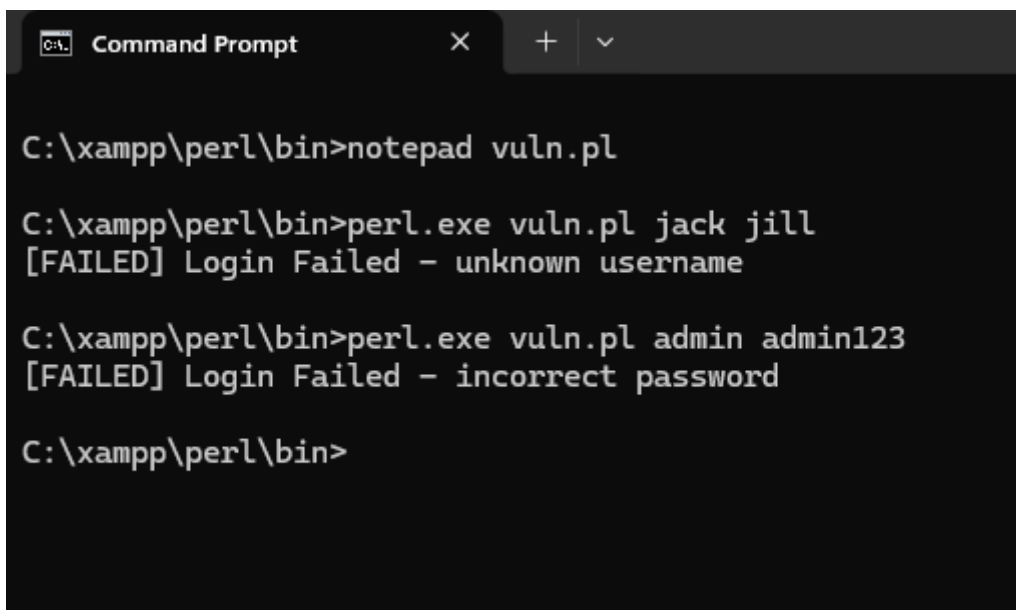
the output is unknown username

run command : perl.exe vuln.pl admin 123

the output is incorrect password , means the username was correct

different messages for when an incorrect username is supplied, versus when the username is correct but the password is wrong.

could allow an attacker to discover a valid username by trying different values until the incorrect password message is returned.



```
C:\xampp\perl\bin>notepad vuln.pl

C:\xampp\perl\bin>perl.exe vuln.pl jack jill
[FAILED] Login Failed - unknown username

C:\xampp\perl\bin>perl.exe vuln.pl admin admin123
[FAILED] Login Failed - incorrect password

C:\xampp\perl\bin>
```

Demonstration 2 :

navigate to xampp/php

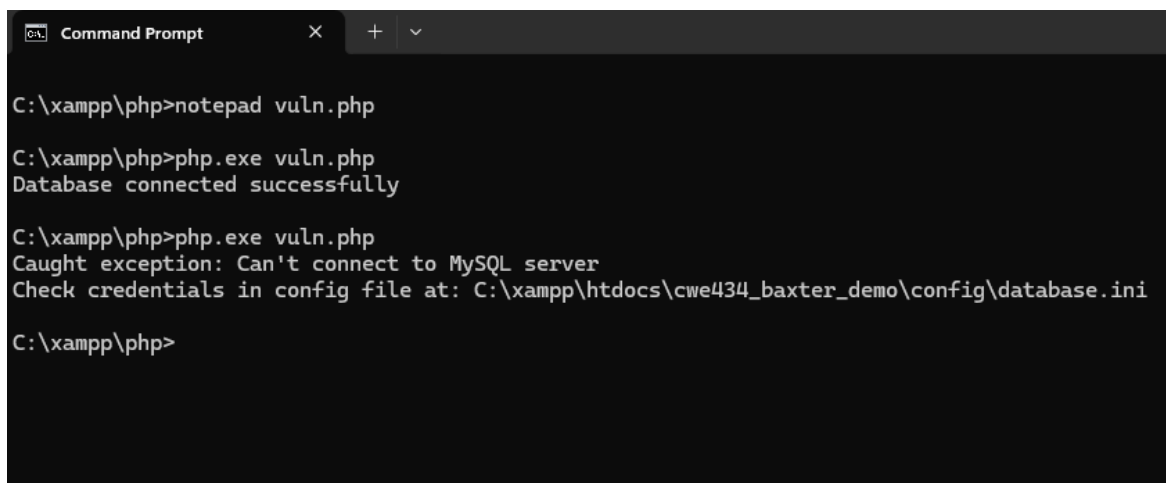
make file vuln.php with this code :

```
<?php
// Simulate database connection
$Mysql_config_location = "C:\\xampp\\htdocs\\cwe434_baxter_demo\\
\\config\\database.ini";

try {
    // Simulate connection failure
    if (rand(0, 1)) { // Randomly fail for demonstration
        throw new Exception("Can't connect to MySQL server");
    }
    echo "Database connected successfully\n";
} catch (Exception $e) {
    // VULNERABLE: Exposes config file location (CWE-200)
    echo "Caught exception: " . $e->getMessage() . "\n";
    echo "Check credentials in config file at: " .
    $Mysql_config_location . "\n";
}
?>
```

run : php.exe vuln.php

run again



```
C:\xampp\php>notepad vuln.php

C:\xampp\php>php.exe vuln.php
Database connected successfully

C:\xampp\php>php.exe vuln.php
Caught exception: Can't connect to MySQL server
Check credentials in config file at: C:\xampp\htdocs\cwe434_baxter_demo\config\database.ini

C:\xampp\php>
```

If an exception occurs, the printed message exposes the location of the configuration file the script is using. An attacker can use this information to target the configuration file (perhaps exploiting a Path Traversal weakness).

Demonstration 3 :

setup openlogic java jdk21

make java file : notepad BankManager.java , with this code :

```
import java.sql.*;

public class BankManager {

    // Simulated method that exposes query in error message
    (CWE-200)
    public void getUserBankAccount(String username, String
accountNumber) {
        String query = null;

        try {
            // Simulate database query (will fail since no real
DB)
            if (username.equals("john")) {
                query = "SELECT * FROM accounts WHERE owner = '" +
username
                    + "' AND accountID = " + accountNumber;

                // This will throw SQLException (no driver, no DB)
                Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/bank");
                Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery(query);
            }

            } catch (SQLException ex) {
                // VULNERABLE: Exposes full query with sensitive
schema information
                String logMessage = "Unable to retrieve account
information from database,\nquery: " + query;
                System.out.println("ERROR: " + logMessage);
                System.out.println("Exception: " + ex.getMessage());
            }
        }

        public static void main(String[] args) {
            BankManager bm = new BankManager();

            // Simulate attack - trigger error with special characters
            System.out.println("=== CWE-200 Information Exposure Demo
===\n");

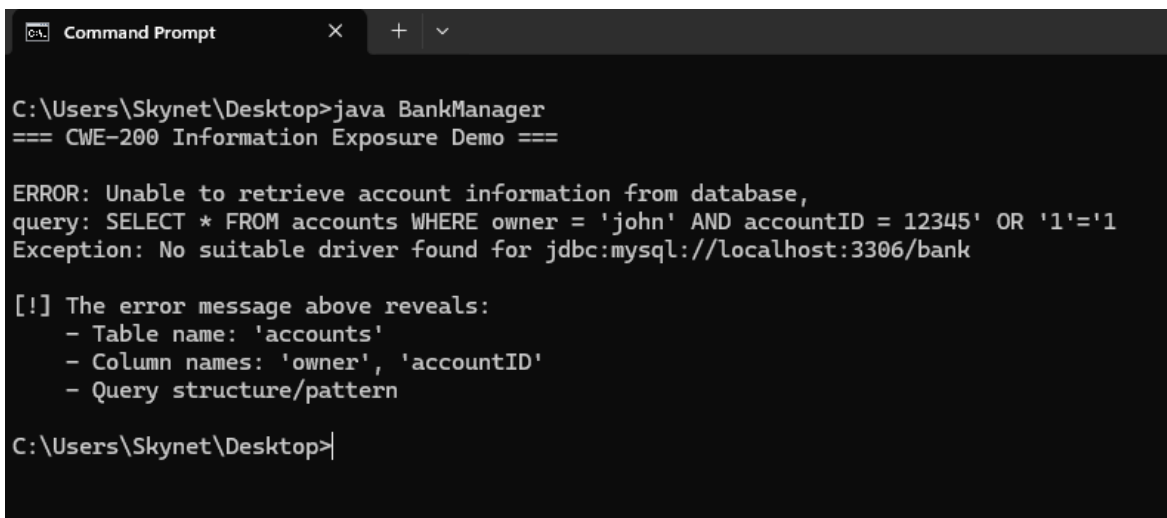
            // Test case that exposes database schema
            bm.getUserBankAccount("john", "12345' OR '1'='1");

            System.out.println("\n[!] The error message above
reveals:");
            System.out.println("        Table name: 'accounts'");
        }
    }
}
```

```
        System.out.println("    - Table name: 'accounts' ),  
        System.out.println("    - Column names: 'owner',  
        'accountID'");  
        System.out.println("    - Query structure/pattern");  
    }  
}
```

compile : javac BankManager.java

run : java BankManager



```
Command Prompt  
C:\Users\Skyne\Desktop>java BankManager  
=== CWE-200 Information Exposure Demo ===  
  
ERROR: Unable to retrieve account information from database,  
query: SELECT * FROM accounts WHERE owner = 'john' AND accountID = 12345' OR '1'='1'  
Exception: No suitable driver found for jdbc:mysql://localhost:3306/bank  
  
[!] The error message above reveals:  
    - Table name: 'accounts'  
    - Column names: 'owner', 'accountID'  
    - Query structure/pattern  
  
C:\Users\Skyne\Desktop>
```

The error message that is created includes information about the database query that may contain sensitive information about the database or query logic. In this case, the error message will expose the table name and column names used in the database. This data could be used to simplify other attacks, such as SQL injection

Demonstration 4 :

from xampp control panel start mysql

navigate to xampp/mysql/bin

run command : mysql -u root -p

```
Command Prompt - mysql -u X + v

C:\xampp\mysql\bin>mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 8
Server version: 10.4.32-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

create test user with some permissions with these commands :

```
CREATE USER 'testuser'@'localhost' IDENTIFIED BY 'Secret123';
GRANT ALL PRIVILEGES ON *.* TO 'testuser'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

```
MariaDB [(none)]> CREATE USER 'testuser'@'localhost' IDENTIFIED BY 'Secret123';
Query OK, 0 rows affected (0.007 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON *.* TO 'testuser'@'localhost';
Query OK, 0 rows affected (0.005 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> EXIT;
Bye

C:\xampp\mysql\bin>
```

try login with wrong credentials and notice the information being revealed in error :

```
C:\xampp\mysql\bin>mysql -u testuser -pWrongPassword
ERROR 1045 (28000): Access denied for user 'testuser'@'localhost' (using password: YES)

C:\xampp\mysql\bin>mysql -u testuser
ERROR 1045 (28000): Access denied for user 'testuser'@'localhost' (using password: NO)

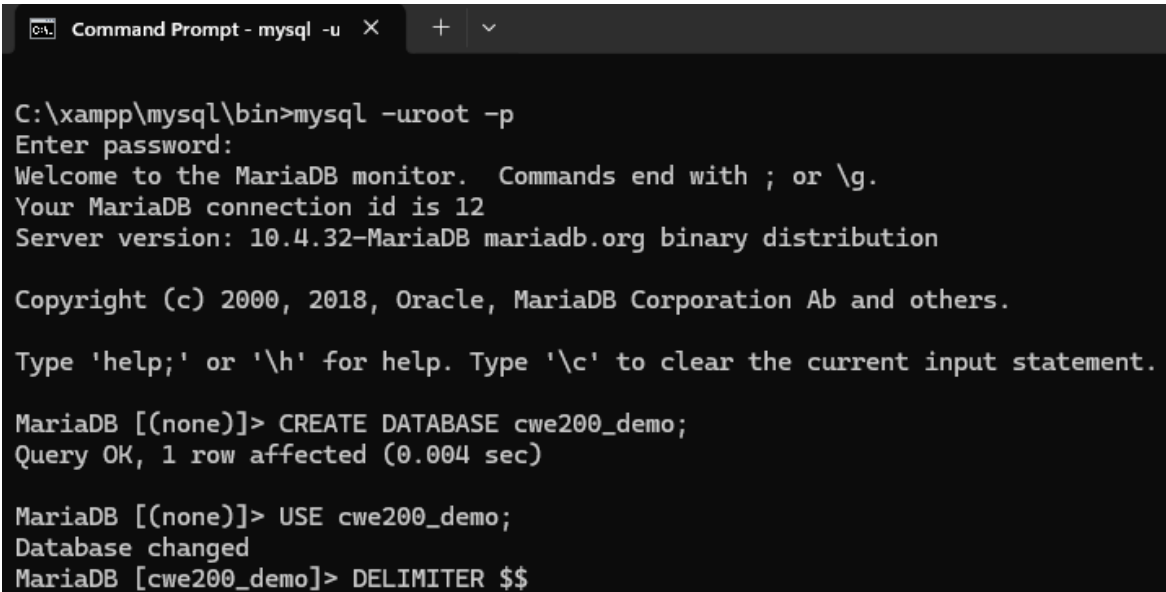
C:\xampp\mysql\bin>mysql -u none -pnone
ERROR 1045 (28000): Access denied for user 'none'@'localhost' (using password: YES)

C:\xampp\mysql\bin>
```

now login to sql again : `mysql -u root -p`

and run :

```
CREATE DATABASE cwe200_demo;  
USE cwe200_demo;  
DELIMITER $$
```



```
Command Prompt - mysql -u  X + v  
  
C:\xampp\mysql\bin>mysql -uroot -p  
Enter password:  
Welcome to the MariaDB monitor.  Commands end with ; or \g.  
Your MariaDB connection id is 12  
Server version: 10.4.32-MariaDB mariadb.org binary distribution  
  
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
MariaDB [(none)]> CREATE DATABASE cwe200_demo;  
Query OK, 1 row affected (0.004 sec)  
  
MariaDB [(none)]> USE cwe200_demo;  
Database changed  
MariaDB [cwe200_demo]> DELIMITER $$
```

```
CREATE PROCEDURE vulnerable_login(IN p_username VARCHAR(50), IN
p_password VARCHAR(50))
BEGIN
    DECLARE actual_password VARCHAR(50);
    DECLARE user_exists INT;
    DECLARE error_msg VARCHAR(200);

    -- This is the actual correct password (what we want to
    expose)
    SET actual_password = 'N1nj4';

    -- Check if user exists in mysql.user table
    SELECT COUNT(*) INTO user_exists FROM mysql.user WHERE User =
    p_username;

    -- Intentionally create an error that exposes the correct
    password
    IF user_exists > 0 THEN
        -- User exists - wrong password? Show error with actual
        password!
        IF p_password != actual_password THEN
            -- Build the error message first, then use SIGNAL
            SET error_msg = CONCAT('Access denied for user: ',
p_username, '@localhost' (Using password: ', actual_password, ')
- Incorrect password provided');
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = error_msg;
        ELSE
            SELECT 'Login successful!' AS result;
        END IF;
    ELSE
        -- User doesn't exist - show generic message
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Access denied
- invalid credentials';
    END IF;
END$$

DELIMITER ;
```

```

MariaDB [cwe200_demo]>
MariaDB [cwe200_demo]> CREATE PROCEDURE vulnerable_login(IN p_username VARCHAR(50), IN p_password VARCHAR(50))
-> BEGIN
->   DECLARE actual_password VARCHAR(50);
->   DECLARE user_exists INT;
->   DECLARE error_msg VARCHAR(200);
->
->   -- This is the actual correct password (what we want to expose)
->   SET actual_password = 'N1nj4';
->
->   -- Check if user exists in mysql.user table
->   SELECT COUNT(*) INTO user_exists FROM mysql.user WHERE User = p_username;
->
->   -- Intentionally create an error that exposes the correct password
->   IF user_exists > 0 THEN
->     -- User exists - wrong password? Show error with actual password!
->     IF p_password != actual_password THEN
->       -- Build the error message first, then use SIGNAL
->       SET error_msg = CONCAT('Access denied for user: ''', p_username, '@localhost'' (Using password: ',
t password provided');
->       SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = error_msg;
->     ELSE
->       SELECT 'Login successful!' AS result;
->     END IF;
->   ELSE
->     -- User doesn't exist - show generic message
->     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Access denied - invalid credentials';
->   END IF;
-> END$$
Query OK, 0 rows affected (0.006 sec)

MariaDB [cwe200_demo]>
MariaDB [cwe200_demo]> DELIMITER ;
MariaDB [cwe200_demo]>

```

```
CALL vulnerable_login('root', 'wrongpassword');
```

```

MariaDB [(none)]> use cwe200_demo
Database changed
MariaDB [cwe200_demo]> CALL vulnerable_login('root', 'wrongpassword');
ERROR 1644 (45000): Access denied for user: 'root@localhost' (Using password: N1nj4) - Incorrect password provided
MariaDB [cwe200_demo]>

```

The error clearly exposes the database credentials.

CWE-497: Exposure of Sensitive System Information to an Unauthorized Control Sphere

Description : The product does not properly prevent sensitive system-level information from being accessed by unauthorized actors who do not have the same level of access to the underlying system as the product does.

Demonstration 5 :

setup mingw gcc g++

notepad vuln.c

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Get the PATH environment variable
    char* path = getenv("PATH");

    // VULNERABLE: Exposing system PATH information (CWE-497)
    fprintf(stderr, "Error: cannot find executable on path: %s\n",
path);

    // Also show current directory for more exposure
    char* cwd = getenv("CD"); // Windows current directory
    if (cwd == NULL) {
        cwd = "C:\\unknown";
    }
    fprintf(stderr, "Current directory: %s\n", cwd);

    // Show system info (additional exposure)
    char* username = getenv("USERNAME");
    char* computername = getenv("COMPUTERNAME");
    fprintf(stderr, "User: %s@%s\n", username, computername);

    return 0;
}
```

compile with gcc : gcc vuln.c -o vuln.exe

run vuln.exe

```

C:\Users\Skyne\Deskto>notepad vuln.c

C:\Users\Skyne\Deskto>gcc vuln.c -o vuln.exe

C:\Users\Skyne\Deskto>vuln.exe
Error: cannot find executable on path: C:\Program Files\OpenLogic\jdk-21.0.8.9-hotspot\bin;C:\WINDOW
S\system32;C:\WINDOWS;C:\WINDOWS\System32\Wbem;C:\WINDOWS\System32\WindowsPowerShell\v1.0\;C:\WINDOW
S\System32\OpenSSH\;C:\MinGW\bin;C:\Program Files\NASM;C:\Users\Skyne\Downloads\GoLink\GoLink.exe;C
:\Program Files (x86)\Windows Kits\10\Windows Performance Toolkit\;C:\Users\Skyne\anaconda3;C:\User
s\Skyne\anaconda3\Library\mingw-w64\bin;C:\Users\Skyne\anaconda3\Library\usr\bin;C:\Users\Skyne\ana
conda3\Library\bin;C:\Users\Skyne\anaconda3\Scripts;C:\Users\Skyne\AppData\Local\Programs\Python
\Python314\Scripts\;C:\Users\Skyne\AppData\Local\Programs\Python\Python314\;C:\Users\Skyne\AppData
\Local\Microsoft\WindowsApps;C:\Exercism
Current directory: C:\unknown
User: Skyne@DESKTOP-MV6IO91

```

Demonstration 6 :

navigate to C:/xampp/php

notepad proc.php

```

<?php
// Simulate getting current username
$username = get_current_user(); // Returns the username of the
script owner

// VULNERABLE: Exposing all processes for the current user
(CWE-497)
echo "=== Processes for user: $username ===\n\n";

// Windows equivalent of 'ps aux' is 'tasklist'
// This shows ALL processes, including command-line arguments
$command = 'tasklist /V /FO LIST';
system($command);

// Additional exposure - show even more details
echo "\n=== Additional Process Details ===\n";
$detailed = 'wmic process get processid,commandline,executablepath
/FORMAT:LIST';
system($detailed);
?>

```

php.exe proc.exe

```
C:\xampp\php>php.exe proc.php  
=== Processes for user: Skynet ===
```

```
Image Name:    System Idle Process  
PID:           0  
Session Name:  Services  
Session#:      0  
Mem Usage:     8 K  
Status:        Unknown  
User Name:     NT AUTHORITY\SYSTEM  
CPU Time:      291:25:16  
Window Title:  N/A
```

```
Image Name:    System  
PID:           4  
Session Name:  Services  
Session#:      0  
Mem Usage:     1,968 K  
Status:        Unknown  
User Name:     N/A  
CPU Time:      5:26:23  
Window Title:  N/A
```

```
Image Name:    Secure System  
PID:           92  
Session Name:  Services  
Session#:      0
```

CWE-214: Invocation of Process Using Visible Sensitive Information

Description : A process is invoked with sensitive command-line arguments, environment variables, or other elements that can be seen by other processes on the operating system.

Demonstration 7 :

notepad SimpleExposure.java

```
public class SimpleExposure {
    public static void main(String[] args) {
        // VULNERABLE: Password comes from command line
        String password = System.getProperty("password");

        if (password == null) {
            System.out.println("No password provided");
        } else {
            System.out.println("Password loaded successfully");
            // In real code, you'd use the password here
        }

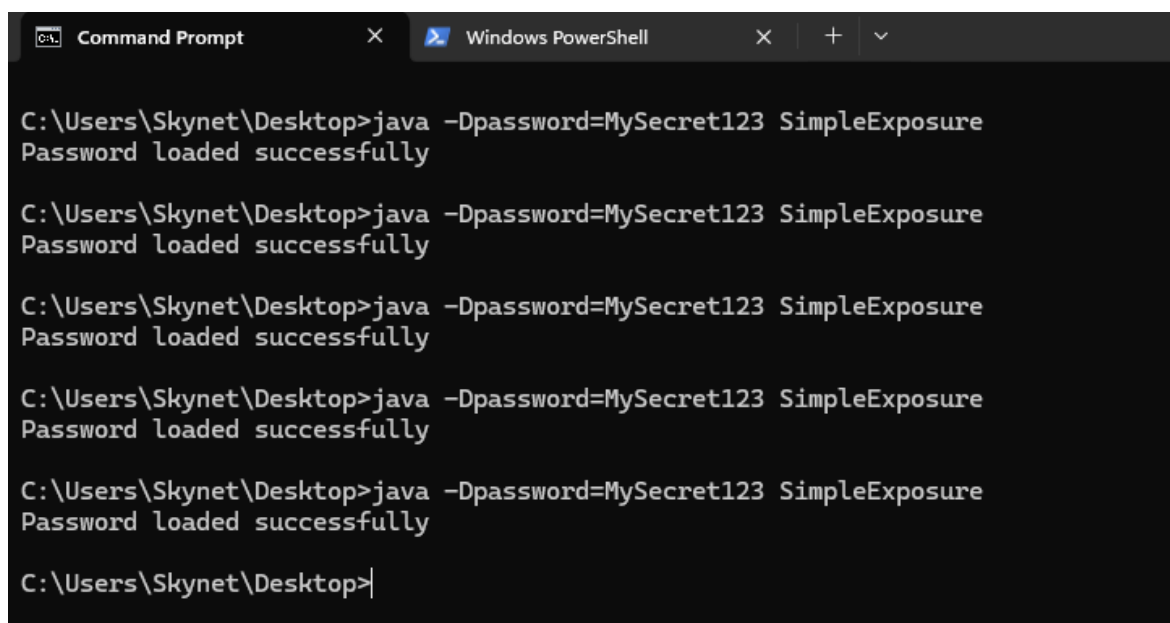
        // Keep program running so we can see it in tasklist
        try {
            Thread.sleep(10000); // Wait 10 seconds
        } catch (InterruptedException e) {
            // Ignore
        }
    }
}
```

compile : javac SimpleExposure.java

run java SimpleExposure

and in another tab run :

Get-WmiObject Win32_Process | Where-Object {\$_.Name -eq "java.exe"} |
Format-Table ProcessId, CommandLine -AutoSize -Wrap



```
Command Prompt x Windows PowerShell x + v

C:\Users\Skynet\Desktop>java -Dpassword=MySecret123 SimpleExposure
Password loaded successfully

C:\Users\Skynet\Desktop>java -Dpassword=MySecret123 SimpleExposure
Password loaded successfully

C:\Users\Skynet\Desktop>java -Dpassword=MySecret123 SimpleExposure
Password loaded successfully

C:\Users\Skynet\Desktop>java -Dpassword=MySecret123 SimpleExposure
Password loaded successfully

C:\Users\Skynet\Desktop>java -Dpassword=MySecret123 SimpleExposure
Password loaded successfully

C:\Users\Skynet\Desktop>
```

```

Command Prompt
Windows PowerShell
PS C:\Users\Skyne> Get-WmiObject Win32_Process | Where-Object {$_.Name -eq "java.exe"} | Format-Table ProcessId, CommandLine -AutoSize -Wrap
ProcessId  CommandLine
-----
185804    java  -Dpassword=MySecret123 SimpleExposure

```

the password for a keystore file is read from a system property.

If the property is defined on the command line when the program is invoked (using the -D... syntax), the password may be displayed in the OS process list.

=====

=====

Previous

CVE-2019-10959 - CWE-434: Unrestricted Upload of File with Dangerous Type

Last updated 0 minutes ago